Doubly Linked List – Insert At Beginning And Time Complexity.

Program :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        newNode->next = nullptr;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }

    else
    {
        newNode->prev = nullptr;
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }

}
```

```
…….
case 1:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtBeginning(data);
            break;
```

Program Analysis:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ ***The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.***

→ ***The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.***

→ ***The size of the Node structure itself is determined by the sum of the sizes of its members***:

→ ***data(an integer) is typically 4 bytes.***

→ ***next(a pointer to another Node) is typically 8 bytes.***

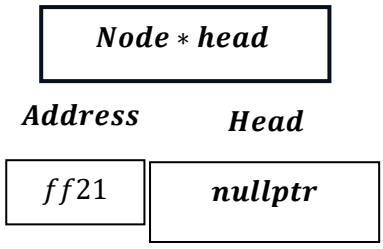
→ ***prev(a pointer to previous Node) is typically 8 bytes.***

→ ***Therefore , the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for prev) .***

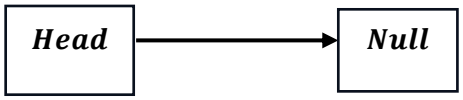
And we already know:

```cpp
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;

Node *head;

void createEmptyList()
{
    head = nullptr;
}
int main()
{
    createEmptyList();
    return 0;


}
```

Head will point to nullptr , at creation of linked list.



i.e.



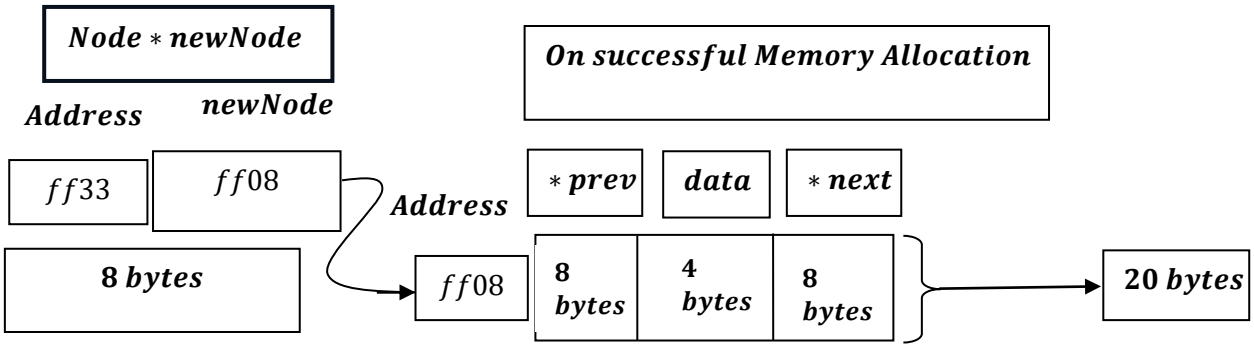
Now coming to insert at beginning :

```
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
```

What does it mean?

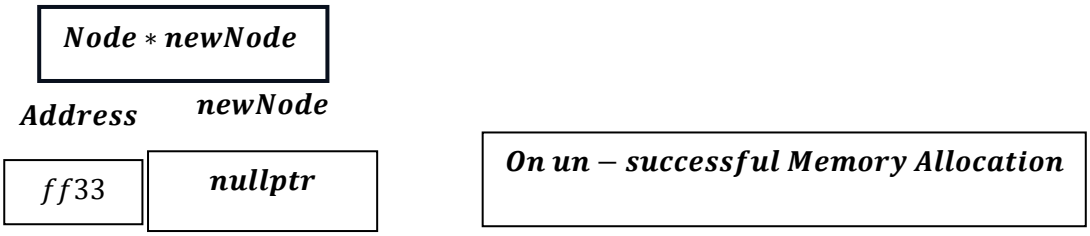
→ newNode will try to allocate memory for data i.e. 4 bytes , next pointer variable i.e. typically 8 bytes, and prev pointer variable that is typically 8 bytes i.e. total 20 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



But on Un − Successful memory allocation:

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



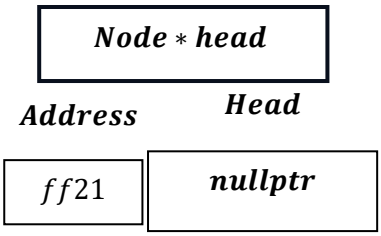
Now, if the value of the pointer variable newNode is nullptr, then:

One get output : `Error: ***Memory Allocation Failed as a message***.`

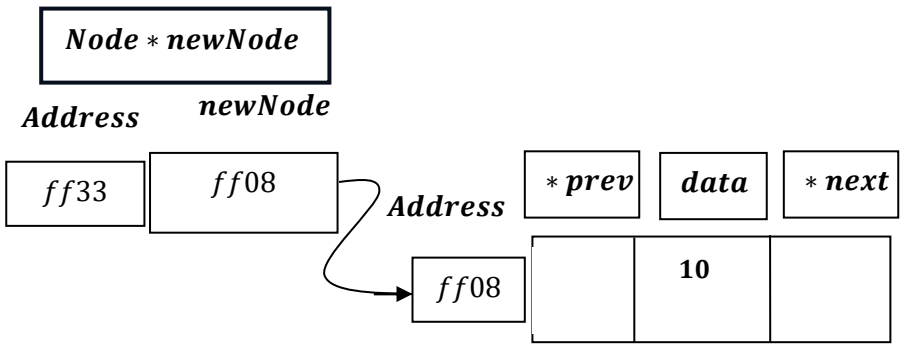
Now,

```
case 1:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtBeginning(data);
            break;
```

Say , data = 10 and currently head is pointed at nullptr.



```
    newNode->data = data;
```

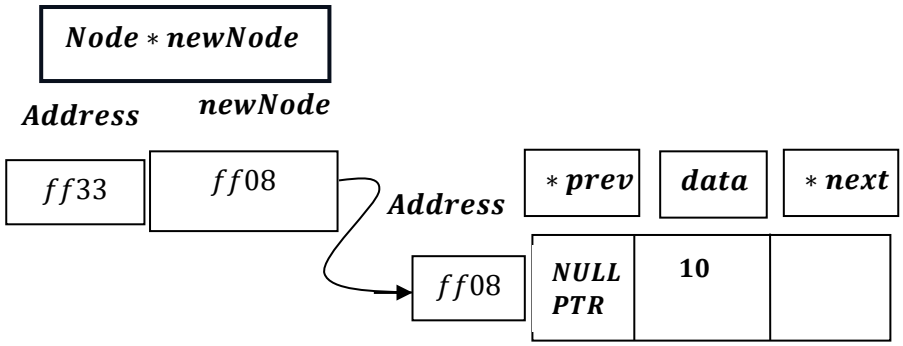


So, head is having value nullpointer or NULL , hence :

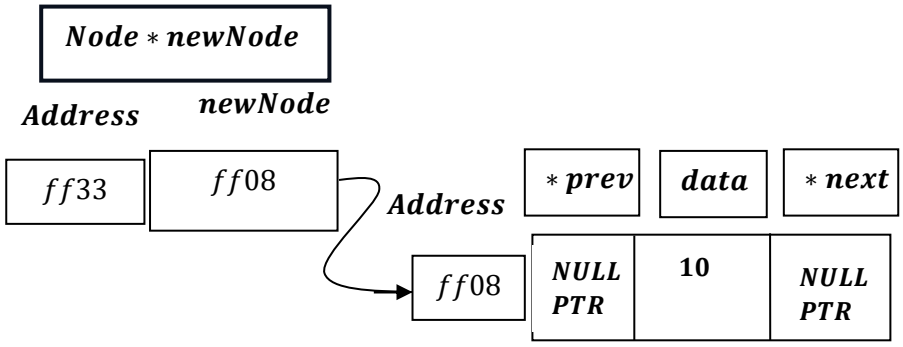
```
if (head == nullptr)
    {
        newNode->prev = nullptr;
        newNode->next = nullptr;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }
```

∴

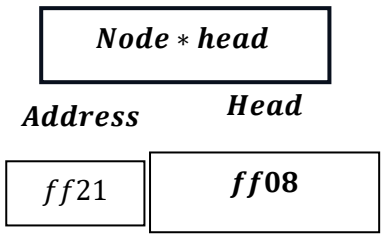
```
        newNode->prev = nullptr;
```



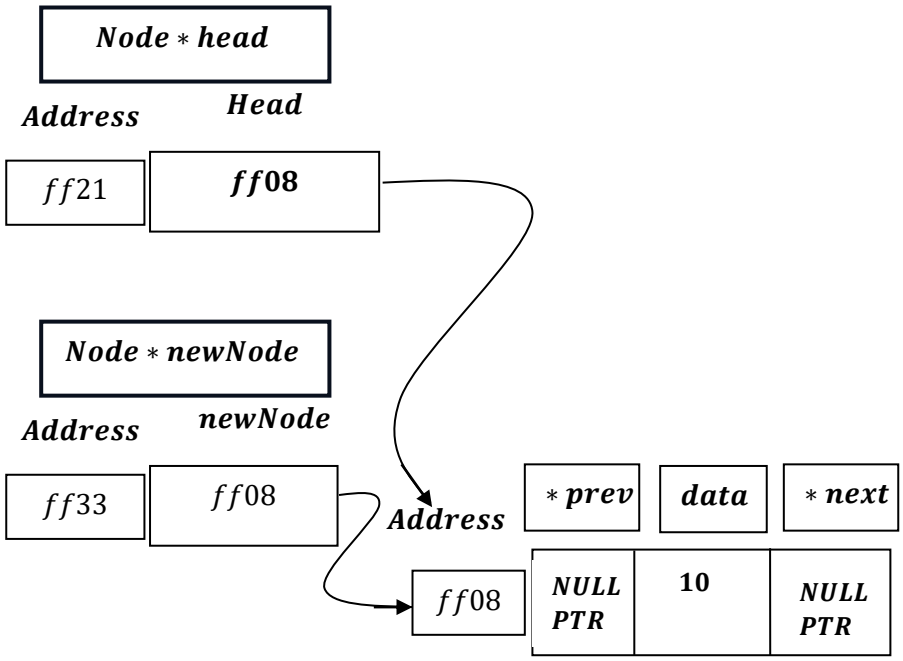

```
        newNode->next = nullptr;
```



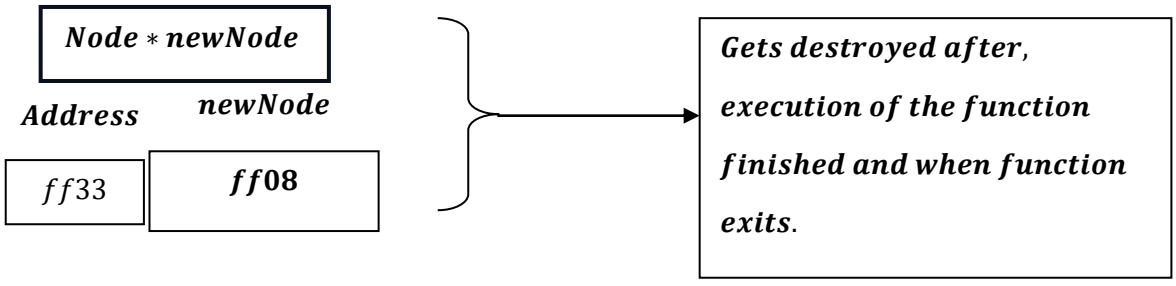

```
        head = newNode;
```



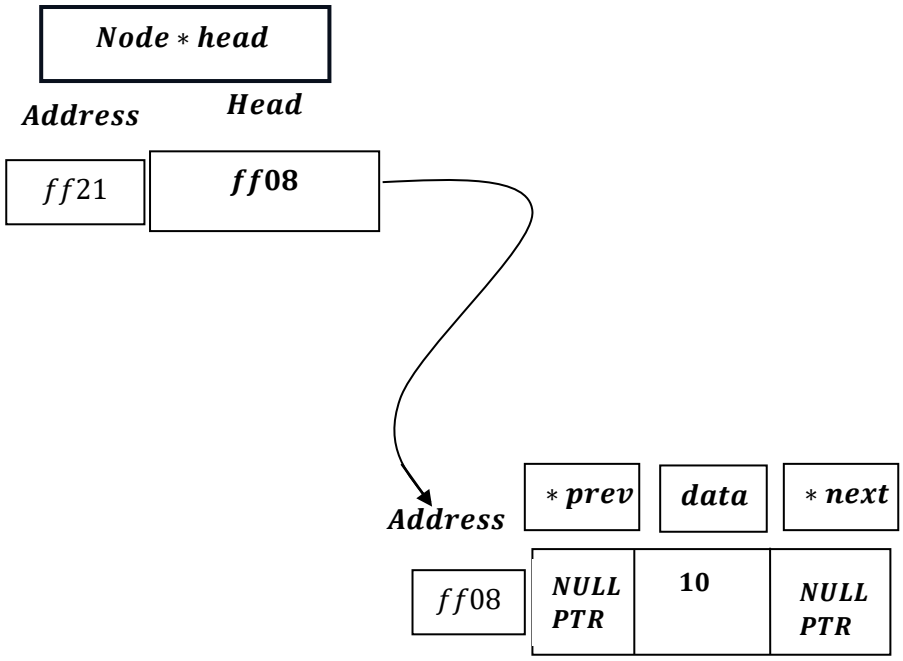
Now , head will point to memory address obtained by pointer newNode.



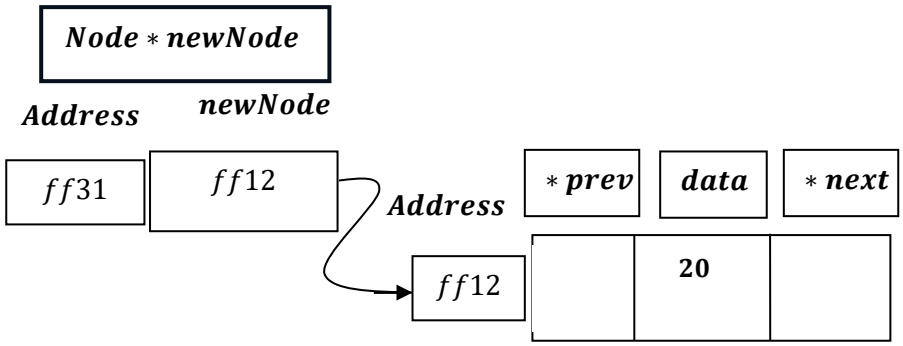
*Now, Node * newNode temporary variable, therefore gets destroyed after function execution get finished .*



Hence , we are left with :



Now say we try to enter data : 20 .



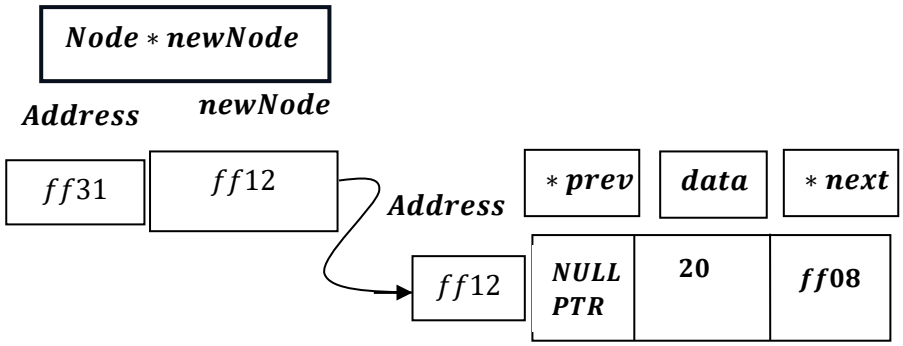
As, head is not having ``nullptr`` , it will now go to else part:

```
    else
    {
        newNode->prev = nullptr;
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
```

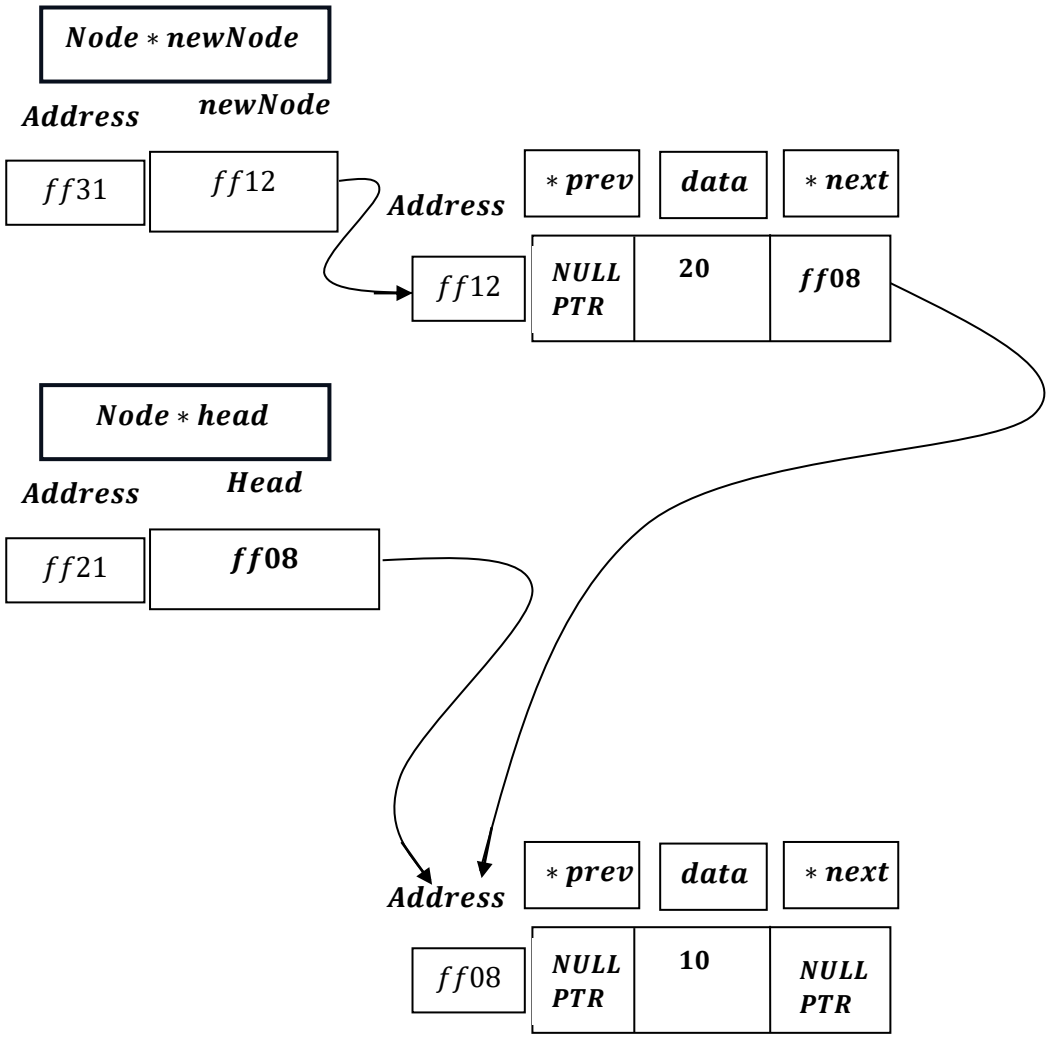
```
        cout << data << " inserted at the beginning." <<
endl;
    }
```

∴

```
        newNode->prev = nullptr;
        newNode->next = head;
```



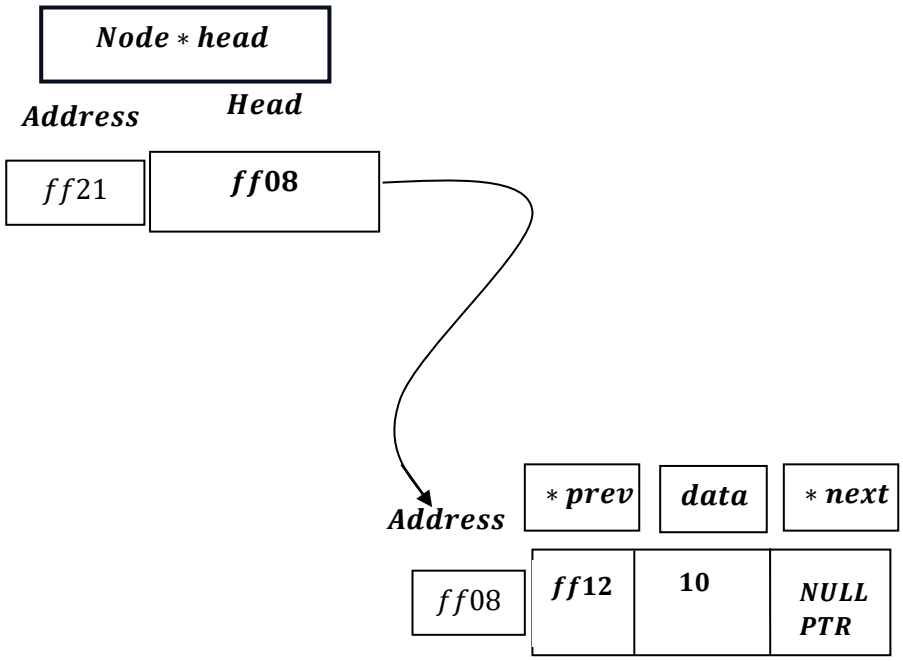
Hence , now , this new node will point to previous Node.



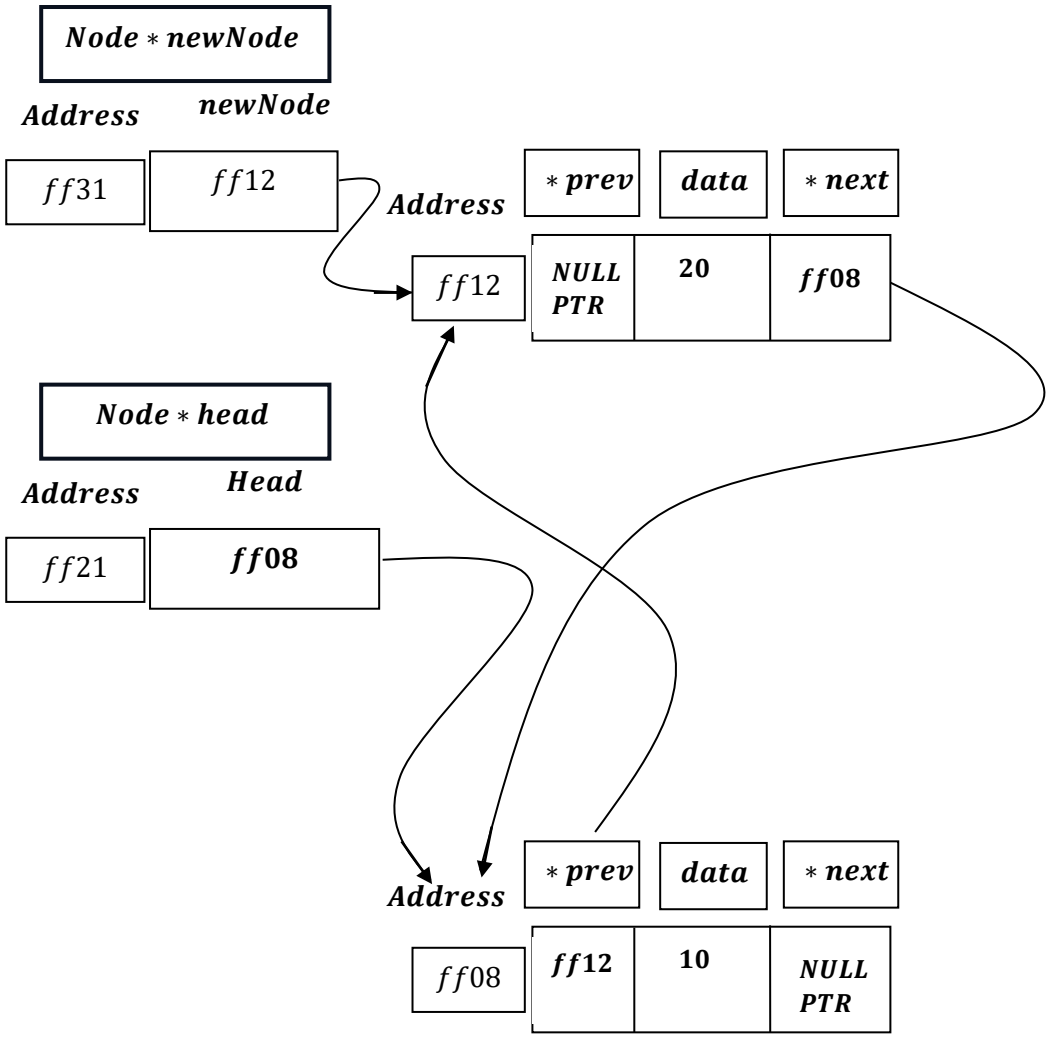
Now,

```
        head->prev = newNode;
```

$i.e., head \rightarrow prev = ff12$;



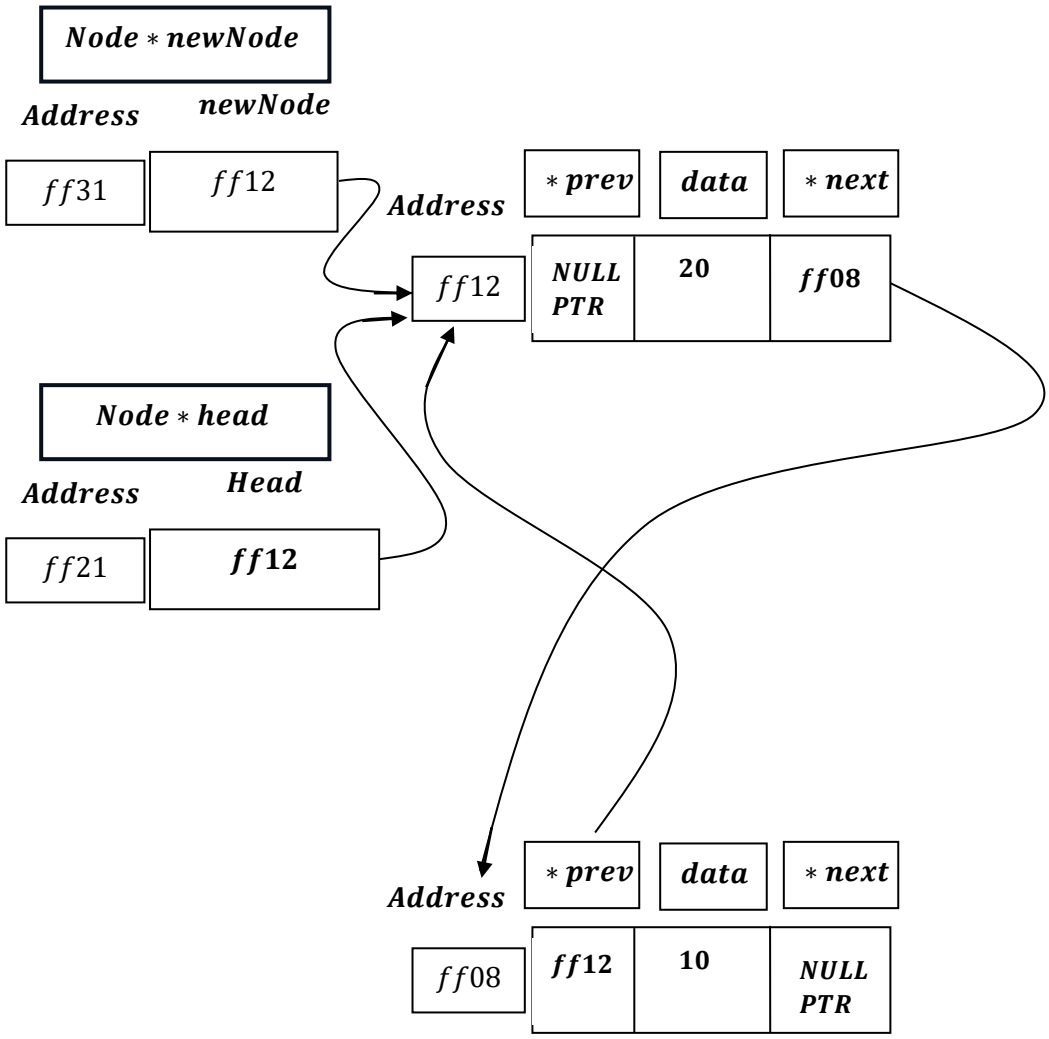
Hence, previous pointer of previous node will now point to current new node.



Next,

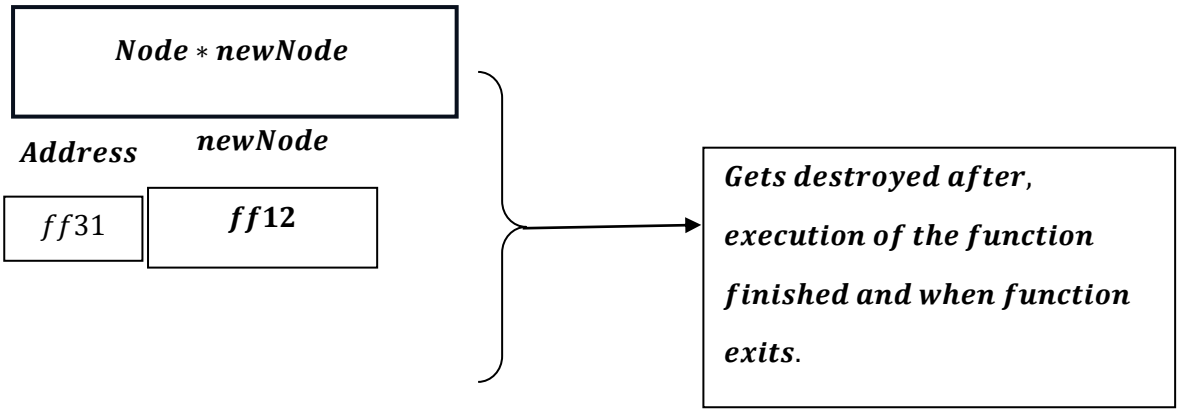
```
        head = newNode;
```

i.e., head = ff12.

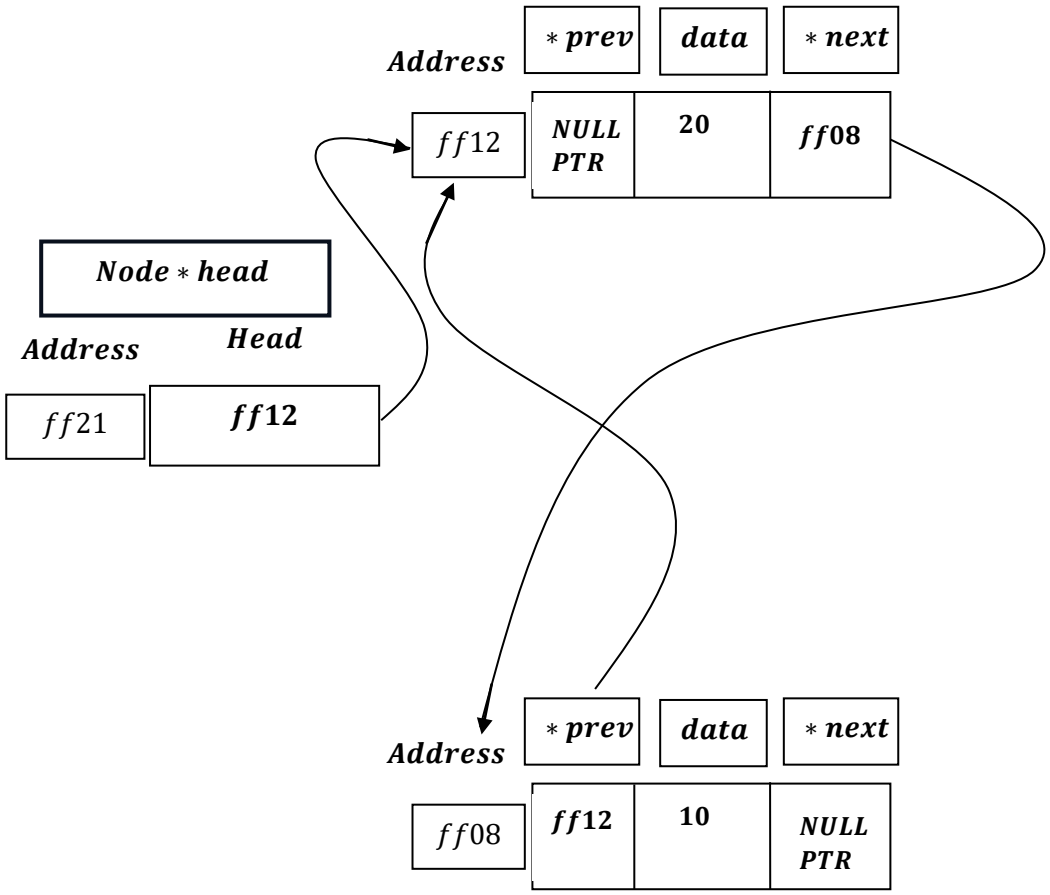


Now, Node * newNode is temporary variable, therefore gets destroyed after function execution get finished .

i.e.



And we are left with:



We can re – write the program as :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = nullptr;
    newNode->next = head;
    if (head != nullptr)
    {
        head->prev = newNode;
    }
    head = newNode;
    cout << data << " inserted at the beginning." << endl;
}
```

Note :

```
if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}
```

This is actually for understanding , hence for optimization , it is removed.

When head == NULL/nullptr , then :

```
Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = nullptr;
    newNode->next = head;
    head = newNode;
}
```

This will run and newNode → next = head = nullptr or NULL .

Except this every condition remain same , when head = NULL or nullptr.

When head ≠ NULL/nullptr , then :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = nullptr;
    newNode->next = head;
    if (head != nullptr)
    {
        head->prev = newNode;
    }
    head = newNode;
}
```

This will run exactly as the else part, of earlier code.

Note, newNode → next = head gets adjusted when head ≠ NULL /nullptr or head = NULL /nullptr, but the process remain same.

Hence , this is more optimized code . than earlier one .

And this process continues as we continue to insert data at bengining in Linked List.

Doubly Linked List - Insert At Beginning [Working Summary]:

→ **If the head pointer is null (i.e., the list is empty)**, the newly created node's `prev` and `next` pointers are both set to `nullptr`. The `head` pointer is then assigned the address of this new node, making it the first and only node in the list.

→ **If the head pointer is not null (i.e., the list is not empty)**, the newly created node's `next` pointer is set to point to the current `head(Node1/first node)` node. The current `head(Node1/first node)` node's `prev` pointer is then updated to point back to the newly created node. Finally, the main `head` pointer is updated to point to the newly created node, officially making it the new head of the list.

Time Complexity of the program

```
void insertAtBeginning(int data)

{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == nullptr)

    {
        cout << "Error: Memory allocation failed." << endl;

        return;

    }


    newNode->data = data;


    if (head == nullptr)

    {
        newNode->prev = nullptr;


        newNode->next = nullptr;


        head = newNode;


        cout << data << " inserted at the beginning." << endl;

    }


    else

    {
        newNode->prev = nullptr;


        newNode->next = head;


        head->prev = newNode;


        head = newNode;

        cout << data << " inserted at the beginning." << endl;

    }


}
```

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

→ *The function allocates memory for a new node using malloc. This operation takes constant time, typically around* $O(1)$ *on most systems.*

→ *The function checks if the new node pointer is null, indicating memory allocation failed. This comparison is a simple operation that takes constant time,* $O(1)$*. Including output statement of if and return statement takes constant time* $O(1)$.

→ *When input integer data value assigned to integer structure member variable `data` through newNode pointer of structure named Node takes constant time* $O(1)$.

→ *Checking pointer head is null takes constant amount of time i.e.* $O(1)$*, which indicates the list is empty, if empty then* :

- *prev pointer of newNode assigned to Null Pointer takes constant time* $O(1)$.
- *next pointer of newNode assigned to Null Pointer takes constant time* $O(1)$.
- *head pointer's value assigned to newNode pointer's value , makes head pointer points to new node which takes constant amount of time* $O(1)$.

→ *Checking pointer head is null takes constant amount of time i.e.* $O(1)$*, if not empty then else part*:

- *prev pointer of newNode assigned to Null Pointer takes constant time* $O(1)$.
- *next pointer of newNode assigned to head pointer's value i.e.*

  now next pointer will point to the address held by head pointer takes constant time $O(1)$.
- *head pointer's value assigned to newNode pointer's value , makes head pointer points to new node which takes constant amount of time* $O(1)$.

Hence if we sum up i.e.

$$O(1) + \Big(O(1) + \big(O(1) + O(1)\big)\Big) + \Big(O(1) + \big(O(1) + O(1) + O(1) + O(1)\big)\Big) = O(1)$$

i.e. Time complexity remains constant .

Doubly Linked List	***Time Complexity***
InsertAtBeginning	$\boldsymbol{O(1)}$

****